Workflow 1 — Analyse du dépôt GitHub avec n8n

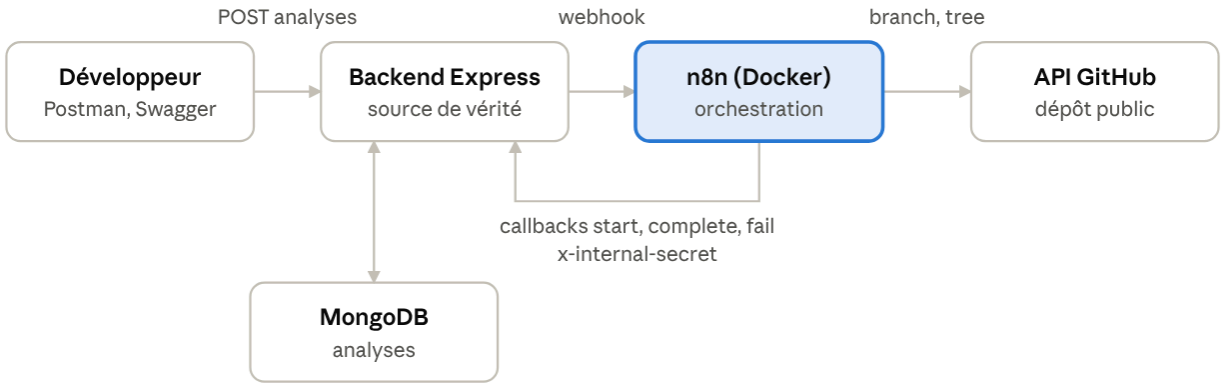
Oct 29, 2026 · @sara

Vue d'ensemble

Le workflow 1 analyse le dépôt GitHub d'un projet : le backend crée une analyse, n8n lit l'arborescence du dépôt, puis renvoie le résultat au backend, qui reste la seule source de vérité.

Le backend répond 202 dès que l'analyse est créée, sans attendre n8n. Le frontend relit ensuite l'analyse (polling) jusqu'à un statut final.

Le backend crée une analyse, n8n la réalise puis rappelle le backend



architecture du workflow 1 · 5 composants

Le développeur ne parle qu'au backend. n8n lit GitHub puis rappelle le backend, qui seul écrit dans MongoDB.

1. Le développeur appelle `POST /api/v1/projects/{projectId}/analyses`.
2. Le backend crée l'analyse en `PENDING`, puis appelle le webhook n8n avec `analysisId` et le dépôt (`owner`, `name`, `branch`).
3. n8n appelle `/start` (`RUNNING`), lit la branche et l'arborescence sur l'API GitHub, puis filtre les fichiers utiles.
4. n8n appelle `/complete` avec le résultat (`COMPLETED`), ou `/fail` en cas d'erreur (`FAILED`).
5. Le développeur lit le résultat avec `GET /api/v1/projects/{projectId}/analyses/{analysisId}`.

Statuts : `PENDING` → `RUNNING` → `COMPLETED` ou `FAILED`. Une transition invalide est refusée avec un 409.

Prérequis

Quatre éléments doivent être prêts avant de commencer.

- **Docker Desktop** installé et démarré, pour faire tourner n8n.
- **Le backend** lancé sur le port 4000, avec son fichier `.env` renseigné (voir l'étape 2).
- **Un projet** appartenant à un développeur approuvé, avec un dépôt GitHub **public** lié et vérifié (`repository.isValid` à `true`).
- **Postman ou Swagger UI** pour appeler l'API. Le dépôt de test utilisé pendant la mise au point est `sarahhamza/fastapi-rds` , branche `main` .

Étape 1 — Lancer n8n avec Docker Desktop

n8n tourne dans un conteneur Docker, accessible sur `http://localhost:5678` . Un volume conserve les workflows et les credentials quand le conteneur est arrêté ou recréé.

1. Démarre Docker Desktop et attends qu'il indique qu'il est prêt.
2. Dans un terminal, crée le volume puis lance le conteneur :

```
docker volume create n8n_data
docker run -d --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n n8nio/n8n
```

3. Vérifie que le conteneur tourne : dans Docker Desktop, l'onglet **Containers** montre `n8n` avec le port `5678:5678` , ou en ligne de commande :

```
docker ps --filter name=n8n
```

4. Ouvre `http://localhost:5678` . Au premier lancement, n8n demande de créer un compte propriétaire, puis affiche l'éditeur.

Commandes utiles pour la suite :

Action	Commande
Arrêter n8n	`docker stop n8n`
Redémarrer n8n	`docker start n8n`
Voir les logs	`docker logs n8n`

Les mêmes actions sont disponibles avec les boutons de l'onglet **Containers** de Docker Desktop.

Sans le volume `n8n_data`, supprimer le conteneur efface les workflows et les credentials. Le conteneur n8n joint le backend, qui tourne sur la machine hôte, avec l'adresse `host.docker.internal` et non `localhost`.

Étape 2 — Configurer le backend

Deux variables de `backend/.env` relient le backend à n8n. Le backend les valide au démarrage et ne démarre pas si elles manquent.

Variable	Rôle	Valeur
`N8N_ANALYSIS_WEBHOOK_URL`	URL du webhook n8n que le backend appelle	`http://localhost:5678/webhook-test/repository-analysis` pendant les tests, `http://localhost:5678/webhook/repository-analysis` en production
`N8N_SHARED_SECRET`	Secret partagé entre le backend et n8n, 32 caractères minimum	Généré avec `openssl rand -base64 32`

Le secret voyage dans le header `x-internal-secret` (constante `INTERNAL_SECRET_HEADER` de `shared/security/internal-auth.ts`). Le backend l'envoie à n8n, et n8n le renvoie sur les routes internes.

Redémarre le backend après toute modification du `.env` : la configuration est lue une seule fois au démarrage.

Routes concernées :

Route	Appelée par	Protection
`POST /api/v1/projects/{projectId}/analyses`	Développeur	JWT Bearer
`GET /api/v1/projects/{projectId}/analyses`	Développeur	JWT Bearer
`GET /api/v1/projects/{projectId}/analyses/{analysisId}`	Développeur	JWT Bearer
`POST /api/v1/internal/analyses/{analysisId}/start`	n8n	`x-internal-secret`

Route	Appelée par	Protection
`POST /api/v1/internal/analyses/{analysisId}/complete`	n8n	`x-internal-secret`
`POST /api/v1/internal/analyses/{analysisId}/fail`	n8n	`x-internal-secret`

Les routes internes ne sont jamais protégées par le JWT utilisateur. Si le secret a été affiché dans une capture ou un message, génères-en un nouveau et mets-le à jour dans `.env` et dans le credential n8n.

Étape 3 — Créer le credential Header Auth

Un seul credential sert à deux usages : n8n vérifie le secret reçu du backend, et il l'envoie au backend sur les routes internes.

1. Dans n8n, crée un workflow nommé `repository-analysis`.
2. Ajoute le nœud **Webhook**. Dans **Authentication**, choisis **Header Auth**, puis **Create new credential** dans la liste du credential.
3. Renseigne le formulaire :
   - **Name** : `x-internal-secret`. C'est le nom du header, pas le titre du credential.
   - **Value** : la valeur de `N8N_SHARED_SECRET` copiée depuis `backend/.env`, sans espace ni retour à la ligne.
4. Enregistre. Tu peux renommer le credential `internal-secret` pour le retrouver facilement.

Les nœuds HTTP Request qui appellent le backend (`start`, `complete`, `fail`) utilisent le même credential : **Authentication** sur **Generic Credential Type**, puis **Header Auth**. Les nœuds qui appellent GitHub n'en ont pas besoin tant que le dépôt est public.

Étape 4 — Construire le workflow n8n

Le workflow enchaîne six nœuds en série : `Webhook → start → branch → tree → filter → complete`. Chaque nœud utilise le résultat du précédent, donc ils ne doivent jamais partir en parallèle depuis `start`.

Pour les nœuds HTTP Request qui appellent le backend : **Method** `POST`, **Authentication** sur **Generic Credential Type**, puis **Header Auth** avec le credential `internal-secret`. Dans leurs URL, `host.docker.internal` désigne le backend, car n8n tourne dans Docker.

1. Webhook

- **HTTP Method** : `POST`.
- **Path** : `repository-analysis`, sans caractère en trop (pas de point final).
- **Authentication** : Header Auth, avec le credential `internal-secret`.
- **Respond** : `Immediately`, pour que le backend reçoive sa réponse sans attendre la fin du workflow.

Le backend envoie `{ "analysisId": "...", "repository": { "owner": "...", "name": "...", "branch": "..." } }`, que n8n place dans `body`.

2. start

Passe l'analyse de `PENDING` à `RUNNING`, sans body. URL :

```
http://host.docker.internal:4000/api/v1/internal/analyses/{{
$json.body.analysisId }}/start
```

3. branch

Method `GET`, sans credential (dépôt public) :

```
https://api.github.com/repos/{{
$('Webhook').first().json.body.repository.owner }}/{{
$('Webhook').first().json.body.repository.name }}/branches/{{
$('Webhook').first().json.body.repository.branch }}
```

La sortie contient `commit.sha`, le SHA du dernier commit de la branche. Il fige l'instantané analysé : les étapes suivantes lisent toutes ce même commit.

4. tree

Method `GET`, sans credential :

```
https://api.github.com/repos/{{
$('Webhook').first().json.body.repository.owner }}/{{
$('Webhook').first().json.body.repository.name }}/git/trees/{{
$json.commit.sha }}?recursive=1
```

Un seul appel renvoie les chemins de tous les fichiers du dépôt. Le champ `truncated` vaut `true` quand le dépôt est trop gros pour être listé en entier.

5. filter

Nœud **Code**, mode **Run Once for All Items**, langage **JavaScript**. Il garde les fichiers utiles à l'analyse (manifestes, Docker, CI), ignore `node_modules`, `dist` et `vendor`, retient au plus 40 fichiers de moins de 200 Ko, et prépare le body de `complete`.

```
const tree = $input.first().json;
const commitSha = $('branch').first().json.commit.sha;

const USEFUL =
  /(^|\/)
(package\.json|requirements[^/]*\.txt|pyproject\.toml|pom\.xml|build\.gradle(
\.kts)?|[^/]+\.csproj|Dockerfile[^/]*|docker-compose[^/]*\.ya?
ml|\.dockerignore)$|^\.github\/workflows\/[^/]+\.ya?ml$/i;
const IGNORED = /(^|\/)(node_modules|dist|vendor|\.git)\//;
const MAX_FILES = 40;
const MAX_SIZE = 200000;

const blobs = tree.tree.filter((entry) => entry.type === 'blob');
const treePaths = blobs.map((entry) => entry.path);

const files = blobs
  .filter(
    (entry) =>
      USEFUL.test(entry.path) &&
      !IGNORED.test(entry.path) &&
      (entry.size ?? 0) <= MAX_SIZE,
  )
  .slice(0, MAX_FILES)
  .map((entry) => ({ path: entry.path }));

const completeBody = {
  detectedFiles: files.map((file) => file.path),
  technologies: [],
  components: [],
  docker: {
    hasDockerfile: treePaths.some((path) =>
/(^|\/)dockerfile[^/]*$/i.test(path)),
    hasCompose: treePaths.some((path) =>
      /(^|\/)docker-compose[^/]*\.ya?ml$/i.test(path),
    ),
    findings: [],
  },
};

return [
```

```
  {
    json: { commitSha, truncated: tree.truncated, treePaths, files,
completeBody },
  },
];
```

Les expressions régulières sont insensibles à la casse : un fichier nommé `dockerfile` est détecté comme `Dockerfile`. Le nom `branch` dans `$('branch')` doit correspondre exactement au nom du nœud.

6. complete

URL :

```
http://host.docker.internal:4000/api/v1/internal/analyses/{{
$('Webhook').first().json.body.analysisId }}/complete
```

L'expression utilise `$('Webhook').first()` et non `.item` : le nœud `filter` produit un élément sans lien avec celui du Webhook, et n8n ne saurait pas lequel choisir.

Pour le body, active **Send Body**, choisis **Body Content Type** `JSON` et **Specify Body** `Using JSON`, puis passe le champ **JSON** en mode **Expression** (icône `fx`) avec :

```
{{ JSON.stringify($json.completeBody) }}
```

Si le champ reste en mode Fixed, n8n lit l'expression comme du JSON brut et affiche « The value in the JSON Body field is not valid JSON ». Pour l'instant, `technologies` et `components` restent vides : ils seront renseignés par l'analyseur (voir la section sur les limites).

Branche d'erreur `fail` (à construire)

Sans cette branche, une erreur dans n8n laisse l'analyse en `RUNNING` et bloque le projet : le lancement suivant renvoie un 409.

1. Pour chaque nœud (`start`, `branch`, `tree`, `filter`, `complete`), ouvre **Settings** et règle **On Error** sur **Continue (using error output)**.
2. Relie chaque sortie **Error** à un unique nœud HTTP Request `fail` : **Method** `POST`, Header Auth, et cette URL :

```
http://host.docker.internal:4000/api/v1/internal/analyses/{{
$('Webhook').first().json.body.analysisId }}/fail
```

3. Dans son body, en mode **Expression** :

```
{{ JSON.stringify({ code: 'WORKFLOW_ERROR', message: $json.error?.message ?? 'Workflow error' }) }}
```

Le backend accepte `fail` depuis `PENDING` et depuis `RUNNING`. Cette branche n'a pas encore été testée : valide-la avec un projet lié à un dépôt inexistant.

Étape 5 — Tester

Teste dans cet ordre : d'abord le backend seul, puis le cycle complet avec n8n.

Test A : les callbacks, sans n8n

Ce test valide le backend et les routes internes. Il sert aussi à débloquer une analyse restée en `PENDING` ou `RUNNING`.

1. Dans Postman, ouvre une requête `POST` `http://localhost:4000/api/v1/internal/analyses/{analysisId}/start`.
2. Onglet **Authorization** sur **No Auth**. Onglet **Headers** : ajoute `x-internal-secret` avec la valeur de `N8N_SHARED_SECRET`.
3. Envoie : tu obtiens un **200** avec `"status": "RUNNING"` et un `startedAt`.
4. Change l'URL pour `.../complete`, onglet **Body** sur **raw** puis **JSON**, avec :

```
{
  "detectedFiles": [],
  "technologies": [],
  "components": [],
  "docker": { "hasDockerfile": false, "hasCompose": false, "findings": [] }
}
```

5. Envoie : tu obtiens un **200** avec `"status": "COMPLETED"` et un `finishedAt`.

Pour abandonner une analyse bloquée, appelle `.../fail` avec le body `{ "code": "MANUAL_RESET", "message": "Reset" }`.

Test B : le cycle complet avec n8n

1. Dans `backend/.env`, mets l'URL de test (`.../webhook-test/repository-analysis`) et redémarre le backend.
2. Dans n8n, clique sur **Execute** pour mettre le Webhook en écoute. L'URL de test ne répond qu'une seule fois par écoute : il faut recliquer avant chaque essai.
3. Dans Postman, lance `POST` `http://localhost:4000/api/v1/projects/{projectId}/analyses` avec ton JWT Bearer. Tu obtiens un **202** et l'identifiant de l'analyse dans `data.id`.

4. Dans n8n, les six nœuds passent au vert dans l'ordre. Ouvre `filter` : son OUTPUT contient `commitSha`, `treePaths`, `files` et `completeBody`.
5. Lance `GET http://localhost:4000/api/v1/projects/{projectId}/analyses/{analysisId}` : le statut est `COMPLETED`, avec `startedAt`, `finishedAt` et `durationMs`.

Sur le dépôt `sarahhamza/fastapi-rds`, un premier essai a renvoyé `detectedFiles: ["requirements.txt"]` en environ une seconde. Le fichier `dockerfile`, écrit en minuscules, n'était pas détecté avant le passage des expressions en insensible à la casse : à revérifier avec un nouvel essai.

Résultats déjà observés

Test	Résultat
`POST` analyse avec n8n qui n'écoute pas, ou URL de webhook erronée	422 `INVARIANT_VIOLATION`, l'analyse passe en `FAILED`
`POST` analyse pendant qu'une autre est `PENDING` ou `RUNNING`	409 `An analysis is already in progress for this project`
`complete` sans body	400 `VALIDATION_ERROR`, `Invalid analysis result`
`start` puis `complete` avec le secret et le body	200, `RUNNING` puis `COMPLETED`
Cycle complet par n8n (URL de test)	202, six nœuds verts, `COMPLETED`

Test C : Swagger UI

Une fois les routes d'analyse ajoutées à `swagger.ts`, clique sur **Authorize** : `bearerAuth` reçoit l'`accessToken` de `/api/v1/auth/login` (sans le préfixe `Bearer`), `internalSecret` reçoit la valeur de `N8N_SHARED_SECRET`. Rejoue ensuite les tests A et B depuis l'interface.

Étape 6 — Passer en production

L'URL de test oblige à cliquer sur **Execute** avant chaque appel. L'URL de production fonctionne en continu, une fois le workflow publié.

1. Dans n8n, clique sur **Publish** (en haut à droite) pour activer le workflow. n8n refuse de publier si un nœud est mal configuré.
2. Dans `backend/.env`, remplace l'URL de test par l'URL de production, sans `-test` :

```
N8N_ANALYSIS_WEBHOOK_URL=http://localhost:5678/webhook/repository-analysis
```

3. Redémarre le backend.
4. Lance une analyse : tu n'as plus rien à cliquer dans n8n. Les exécutions de production n'apparaissent pas sur le canvas, mais dans l'onglet **Executions** du workflow.

Le conteneur n8n doit rester démarré : si Docker Desktop est fermé, le lancement d'une analyse renvoie un 422 et l'analyse passe en `FAILED`. Après chaque modification du workflow, vérifie qu'elle est bien publiée avant de tester l'URL de production.

Dépannage

Presque toutes ces erreurs ont été rencontrées pendant la mise au point du workflow. L'onglet **Executions** de n8n et le message de l'OUTPUT du nœud en erreur donnent la cause exacte.

Symptôme	Cause	Solution
n8n répond 404, `webhook ... is not registered`	Le workflow n'est pas publié (URL `/webhook/`), ou n8n n'écoute pas (URL `/webhook-test/`)	Publier le workflow, ou cliquer sur **Execute** juste avant l'appel
`POST` analyse : 422 `Analysis workflow rejected the request (404)`	Même cause, ou URL du `.env` erronée (point final, `-test` en trop ou manquant)	Corriger `N8N_ANALYSIS_WEBHOOK_URL`, redémarrer le backend
L'URL de test se termine par un point	Le champ **Path** du Webhook contient `repository-analysis.`	Retirer le point final
n8n répond 403 `Authorization data is wrong!`	Header `x-internal-secret` absent, ou différent de celui du credential	Vérifier **Name** et **Value** du credential. Dans Postman : **No Auth** et ajouter le header
`start` : 404 `No route for POST /api/v1/internal/analyses//start`	`analysisId` vide : le Webhook affiche une ancienne donnée sans body, ou le `/` avant `start` manque dans l'URL	Retirer l'épingle du Webhook, **Execute**, relancer un `POST` ; vérifier l'URL
`POST` analyse : 409 `An analysis is already in progress`	Une analyse est restée `PENDING` ou `RUNNING` après une erreur	La terminer avec `/complete` ou `/fail` (Test A)
`complete` : 400 `Invalid analysis result`	Body absent ou mal formé	Body **raw** puis **JSON** dans Postman

Symptôme	Cause	Solution
n8n : `The value in the JSON Body field is not valid JSON`	Le champ **JSON** est resté en mode Fixed	Passer le champ en mode **Expression** (`fx`)
Triangle orange sur `complete`	`$('Webhook').item` n'a pas de lien avec l'élément produit par `filter`	Utiliser `$('Webhook').first()`
Un nœud n'a pas son entrée, ou tous partent de `start`	Les nœuds sont reliés en parallèle	Relier en série : `start → branch → tree → filter → complete`
`hasDockerfile` reste à `false`	Fichier nommé `dockerfile` , ou non poussé sur GitHub	Expressions insensibles à la casse (voir `filter`), puis `git push`
n8n ne joint pas le backend	`localhost` désigne le conteneur n8n	Utiliser `host.docker.internal` dans les URL

Limites actuelles et prochaines étapes

Le workflow fait aujourd'hui le trajet complet, mais le résultat reste partiel : il liste les fichiers utiles et détecte la présence de Docker, sans lire leur contenu.

Limites connues :

- `technologies` et `components` sont envoyés vides à `complete` : la stack (FastAPI, base de données, etc.) n'est pas encore détectée.
- Aucun `finding` Docker n'est produit : le Dockerfile et le `docker-compose` ne sont pas encore vérifiés.
- Seuls les dépôts publics sont pris en charge. Sans token, l'API GitHub est limitée à 60 requêtes par heure et par adresse IP, et une analyse en consomme deux (`branch` et `tree`).
- Le contenu d'un dépôt est une donnée non fiable : l'analyseur devra le lire sans jamais l'exécuter.
- Les valeurs de `category` , `type` et `severity` envoyées à `complete` doivent appartenir aux listes de `domain/enums/analysis.enums.ts` , sinon le backend refuse le résultat.

Prochaines étapes :

- [ ] Construire la branche d'erreur `fail` et la valider avec un dépôt inexistant.
- [ ] Télécharger le contenu des fichiers retenus (Split Out, puis HTTP Request sur `raw.githubusercontent.com` au `commitSha` figé, puis Aggregate).
- [ ] Écrire l'analyseur : module TypeScript du backend ou service Python séparé, avec le même contrat d'entrée et de sortie.

- [ ] Renseigner `technologies`, `components` et les `findings` Docker dans le body de `complete`.
- [ ] Ajouter au modèle `Analysis` le champ `commitSha` et les informations sur les pipelines CI.
- [ ] Passer aux workflows 2 (propositions de l'IA) et 3 (application par Pull Request).